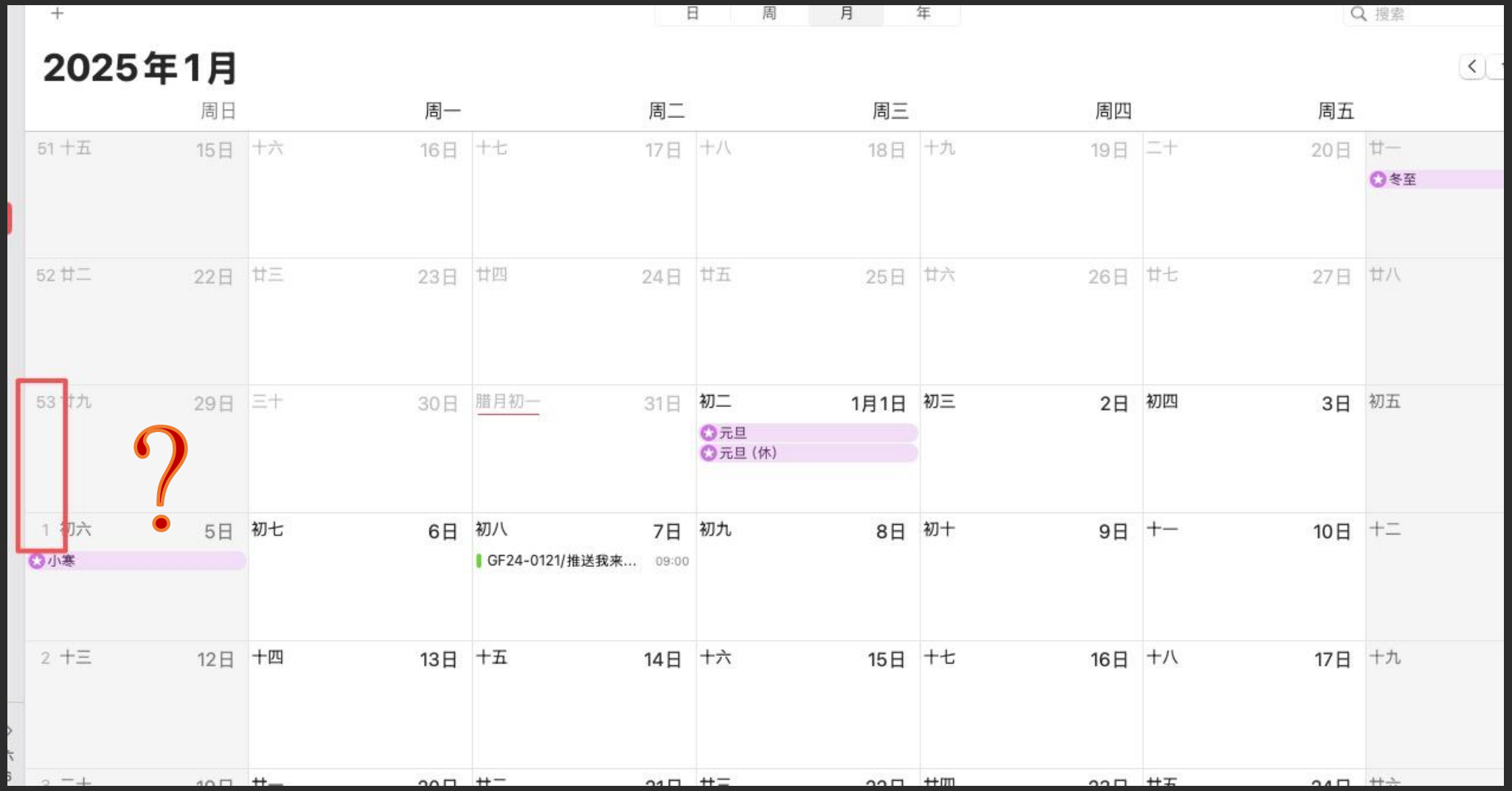
从使用到实现

前端时间工具库的原理拆解

分享者：[贺迎春]

2026.04

开场：今天我们先看一张图



开场：今天我们不讲什么？

我其实不太想讲这些“老生常谈”的内容：

Moment.js 怎么用

作为老牌的时间处理库，它的 API 调用方式在各种文档和教程中随处可见，大家基本都已熟练掌握。

Day.js 怎么用

作为 Moment.js 的现代轻量化替代方案，其 API 设计高度一致，相关的入门资料也非常丰富。

因为 —— 这些，大家其实都会。

开场：我们真正关心的问题

核心追问

我更想先回答一个问题：

时间工具库，到底在帮我们解决什么本质问题？

主流溯源

以及，我们深度思考：

为什么 dayjs 会在众多竞品中，成为今天的主流选择？

实践验证

最后，我们要一起做一件事：

亲手实现一个 mini 时间库，用代码来验证这些设计思想。

CONTENTS

01 为什么“时间”这么难？

探索时间处理中的时区、格式与计算痛点

02 moment：它解决了什么问题？

回顾经典库的设计初衷与核心API能力

03 dayjs：它是怎么重写这套设计的？

Immutable 架构与模块化设计的演进思路

04 我们自己实现一个时间库

从零构建核心类，理解时间戳与格式化原理

05 一些有意思的差异 (Q&A)

探讨不同库在边界场景下的行为差异

06 总结与未来展望

现代前端时间处理的最佳实践与趋势

01

为什么“时间”这么难？

CHAPTER ONE

时间处理的日常与陷阱

前端开发中的高频时间操作场景

展示时间 Format

将时间戳或 Date 对象转换为用户可读的字符串格式，是最基础也是最高频的操作。

计算时间 Calc

计算时间的差值（如：两个日期相隔多少天），或进行时间的增减（如：30分钟后）。

解析时间 Parse

将字符串（如 "2024-05-20"）或接口返回的非标格式转换为标准的 Date 对象进行处理。

你必须知道的核心陷阱

时间处理**不是**一个“纯数学计算问题”，而是一个需要处理时区、历法、夏令时的“复杂规则执行问题”。

经典的坑与时区问题

一个经典的时间解析陷阱

```
new Date('2024-03-01') // 结果因引擎而异
```

标准未明确该格式的解析规则，不同 JS 引擎（V8/JavaScriptCore）对其是 'UTC' 还是 '本地时间' 的判定存在差异。

时区问题的核心逻辑

同一个 UTC 时间点，在不同地区的本地显示截然不同

- 统一基准：UTC 是全球统一的时间参考，无偏移量
- 本地时间：基于 UTC 加上地区偏移（如中国 UTC+8）
- 夏令时 (DST)：部分地区人为调快1小时，增加处理复杂度

```
> new Date('2024-03-01')
< Fri Mar 01 2024 08:00:00 GMT+0800 (中国标准时间)
> new Date('2024-03-01')
< Thu Feb 29 2024 16:00:00 GMT-0800 (北美太平洋标准时间)
```

Date 的本质与小结

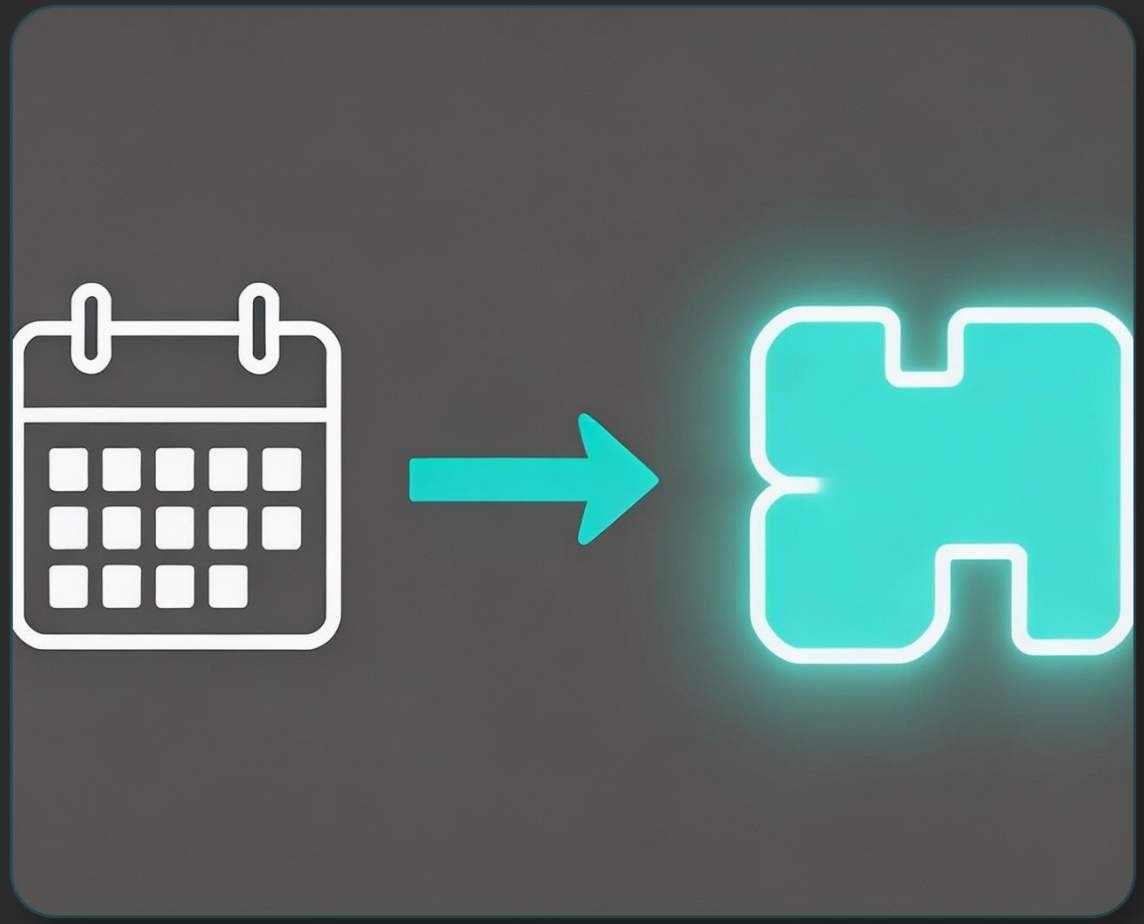
date-js-demo.js

JS 的 Date，本质是？

从 1970-01-01 UTC 开始的 **毫秒数 (Timestamp)**

Date 不关心“年月日”

只关心一个数字



核心认知总结

时间的问题，从来不是“计算逻辑”有多难，而是**“人类定义它的规则太复杂”**。

02

moment

它解决了什么问题?

CHAPTER 02

moment 的核心价值与设计问题

moment-dayjs-demo.js

核心价值：把复杂的时间操作“封装化”

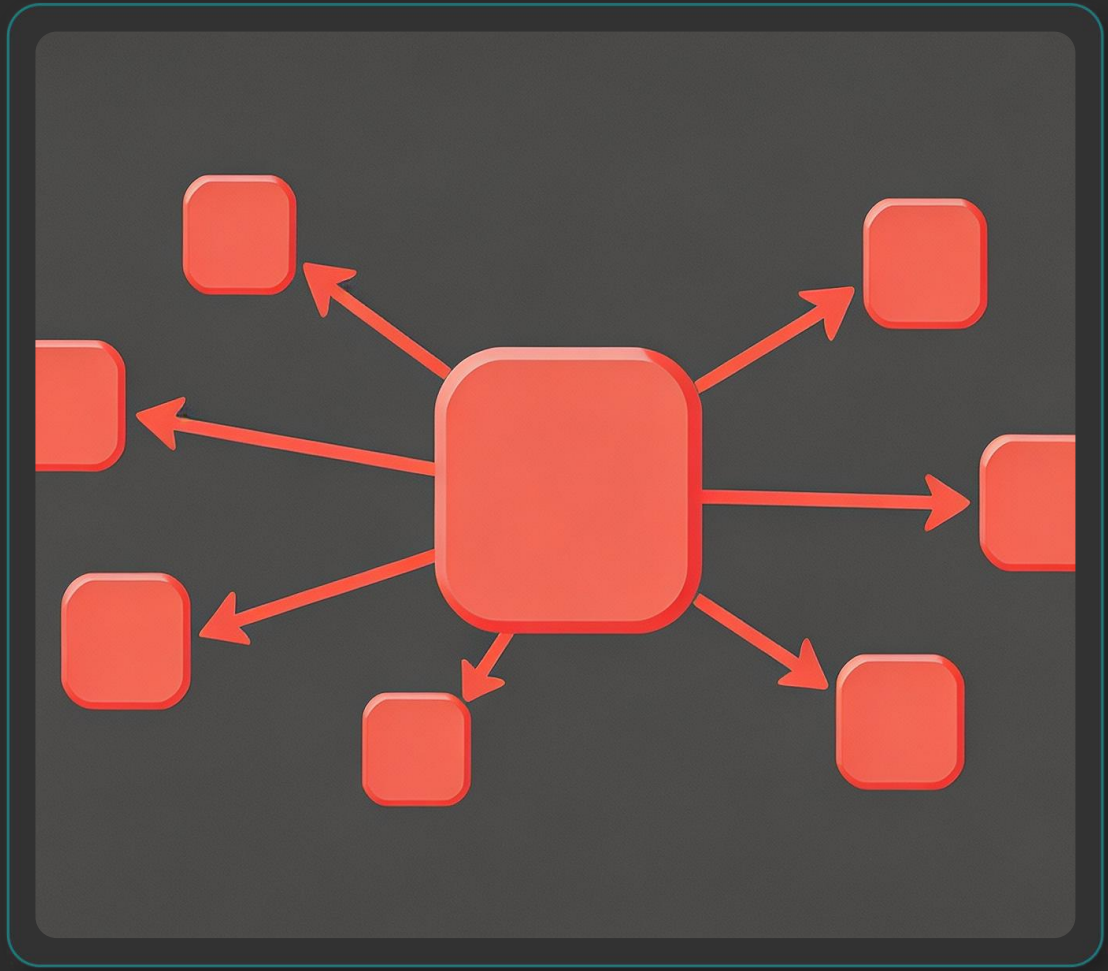
通过统一的 API 屏蔽原生 Date 的底层复杂性

- 原生 Date 难用 → 内部封装，对外暴露极简接口
- 时间格式化繁琐 → 内置 format 方法统一标准

设计缺陷：Mutable (可变对象模型)

```
const m = moment(); m.add(1, 'day'); //    直接修改了原始对象 m
```

在复杂业务中，这种**隐式的状态修改**会导致数据流向不可预测，极易引发难以排查的隐蔽 Bug。



moment 的体积问题与总结

第二个核心痛点：体积冗余

全量功能强制打包

将时区、多语言、解析、格式化等所有功能耦合在一起，无法进行按需拆分引入。

不支持 Tree-shaking 优化

输出格式为 CommonJS，未采用 ESM 模块化标准，导致构建工具无法剔除无用代码。

本质问题：缺乏现代化的模块化架构设计

核心价值与时代局限总结

历史功绩：解决了“能不能用”的问题

统一了早期 JS 社区混乱的日期处理标准，提供了完整的解析、格式化、计算 API，完成了从 0 到 1 的基础建设。

明显短板：没解决“如何优雅用”的问题

结论：适合快速原型开发，不推荐现代大型工程

CHAPTER 03

dayjs：它是怎么重写这套设计的？

dayjs 的三大设计思想

01 / 如何避免副作用?

Immutable (不可变对象) 模式

```
const d2 = d1.add(1, 'day')
// d1 保持不变，返回全新的 d2 对象
```

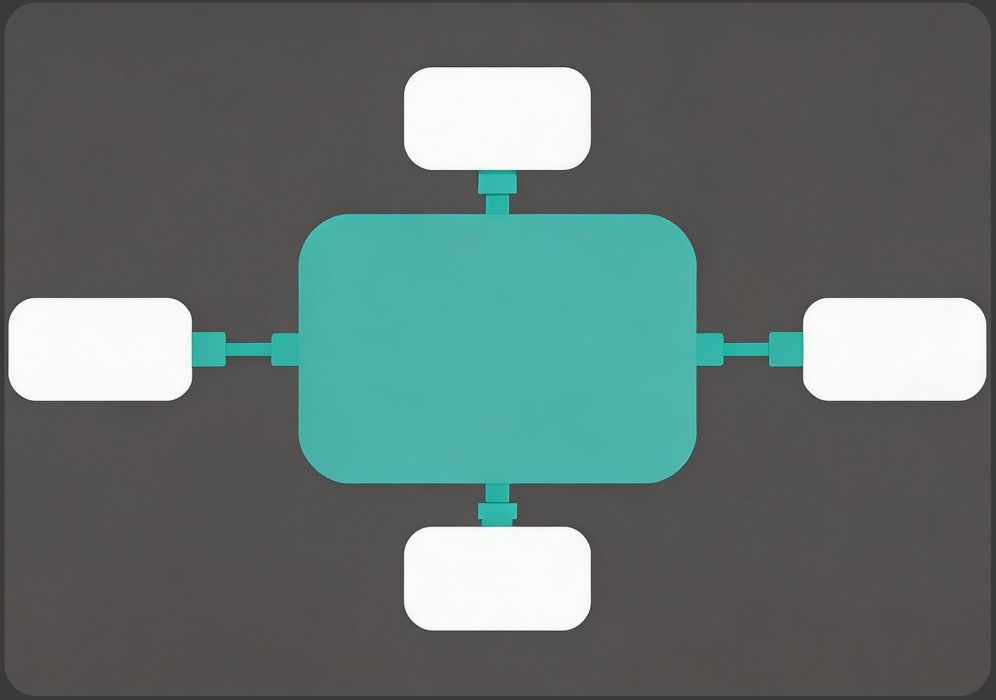
核心思想：拥抱函数式编程，数据操作无副作用

02 / 如何解决体积问题?

按需加载的插件系统 (Plugin System)

```
dayjs.extend(relativeTime) // 仅加载需要的功能
```

核心设计：核心库极致轻量化，功能扩展灵活化



Architecture Design

Core + Plugins = 极致的灵活性与极小的体积

dayjs 的使用体验与总结

03 / 如何保证极致体验?

核心特性：链式调用 (Chaining)

```
dayjs().add(1, 'day').startOf('month').format('YYYY-MM-DD')
```

底层逻辑：每次操作返回**新的实例**，而非修改原对象。这让链式调用既安全又直观。

核心设计思想总结

dayjs 本质是：用**“函数式 + 插件化”**的现代架构，重写了一遍 Moment.js。

带来的核心优势：体积极小 (2KB) · 完全按需引入 · 保持 API 100% 兼容

04

CORE IMPLEMENTATION

我们自己实现一个时间库

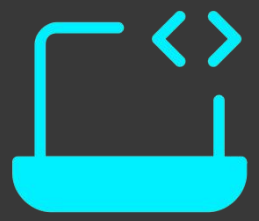
第四部分：我们自己实现一个时间库

有意思的挑战

告别枯燥的源码阅读，我们将亲手从零构建一个**简化版时间库**，把理论转化为实际代码。

核心目标

重点不在于“写了多少功能”，而是深度理解并**复刻它的设计思想**，理解优秀库的架构逻辑。



CODE & THINK

如果我们能用**几十行代码**就把 DayJS 的核心逻辑复刻出来，就说明我们真正**理解**了它的设计精髓，而不再是仅仅停留在“会调用 API”的表层阶段。

第一步：工厂 + 解析

「各种输入 → 只一个 Date」

01 / 统一输入标准

无论外部传入的是 String、Number 时间戳、原生 Date 实例还是 Day.js 对象，工厂函数都会进行统一解析，最终转化为一个标准的 **Date**对象。

核心原则：**外部世界是混乱的，但内部必须统一。**
屏蔽输入的多样性，让核心逻辑只处理确定的标准数据。

02 / 只存一份核心状态

```
class Dayjs {
    constructor(d) {
        this.$d = d; // 唯一真相：只有这一个 Date 对象
    }
}
```

架构哲学：**解析归工厂，真相归 $d。**
所有后续的时间操作、计算、格式化，都是围绕实例中这唯一的 Date 对象展开。

第二步：薄内核 (1/2)

「读用工具、写分两类」—— DayJS 核心设计哲学

展示 (Format) 怎么做?

本质是一个**“模板替换引擎”**。
将复杂的格式化逻辑完全拆解到独立的工具函数中，通过 Token 扫描与正则表达式匹配替换，实现对任意时间格式的动态、灵活渲染。

时间计算 (Add) 怎么做?

固定单位 (秒 / 分 / 时 / 天)

计算逻辑简单直接，基于**毫秒数**进行基础算术运算即可完成。

日历单位 (月 / 年 / 季度)

无法用毫秒直接推算，需结合真实日历规则（如大小月）进行特殊边界处理。

例如月份的加减法：先对齐到当月 1 号，再改到目标月，最后用 min(原来的「日」, 目标月天数) 设日。

第二步：薄内核 (2/2)

日历单位处理

处理月、年等单位时，dayjs 使用专门函数（如 addCalendarMonths），通过“先设为1号、跳月、再取最小天数”的核心规则来精准计算，避免毫秒计算的误差。

“固定单位走毫秒，日历单位走规则”

不可变 (Immutable)

add / subtract 等操作永远不会修改原始对象，而是返回一个包含新 Date 的实例。这是 Dayjs 避免副作用、保证数据纯净的根本原因。

```
add(value, unit) {
    const base = cloneDate(this.$d); // 复制原日期
    // ... 修改 base ...
    return new Dayjs(base); // 返回新实例
}
```

第三步：插件机制

「能力外挂，而不是内置」

◈ 核心原理：原型挂载

```
// 本质是扩展原型链
Dayjs.prototype.fromNow = function() {
  return formatRelative(this.valueOf(),
Date.now(), { nowLabel: '刚刚' });
}
```

将能力“挂载”到核心对象的原型上，实现了功能的灵活扩展。

按需注入 · 极致轻量

核心库体积保持最小化，所有非基础功能（如相对时间、本地化）均通过插件按需引入，避免无用代码导致的性能负担。

职责解耦 · 架构清晰

核心层仅专注于“时间对象”与“基础运算”的维护；所有业务相关的复杂能力，全部通过统一的 extend 接口进行注入与管理。

总结：Dayjs 插件机制以“最小内核 + 无限扩展”的设计理念，完美平衡了体积与功能性。

总结与过渡

统一输入

利用工厂模式封装输入逻辑，屏蔽底层差异，实现统一的调用接口。

内核组织

构建精简的薄内核核心，专注于核心调度，将具体功能逻辑向外剥离。

插件扩展

采用插件化架构，按需加载功能模块，实现工具库的轻量与灵活扩展。

我们复刻的不仅是代码，更是一套核心设计思想：

“如何在一个复杂领域里，设计一个‘小而强’的工具库”

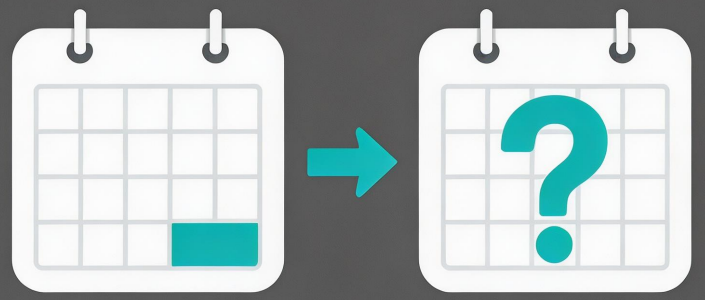
接下来，让我们通过实战演示，来直观感受一下我们的 Mini 版本与 Day.js 在核心细节上的差异与优势。dayjs的插件有很多。我们只是做了一个。

05

INTERESTING DIFFERENCES

一些有意思的差异 (Q&A)

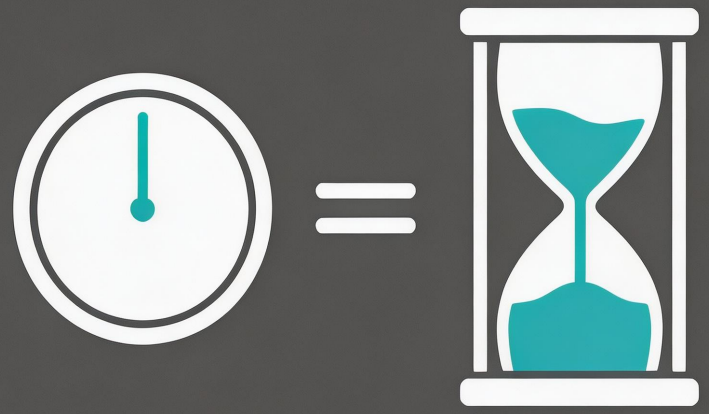
Q1 & Q2



为什么 1 月 31 + 1 个月不对?

- **原因**：JS Date 采用“溢出计算”逻辑，会得到 3 月 3 日。
- **Dayjs**：先定位到目标月份，再取该月最小合法天数（2月28/29日）。

总结：时间库在“纯计算规则”和“人类直觉认知”之间做了取舍。



119 秒为什么会显示为“2 分钟”?

- **Dayjs 逻辑**：在相对时间展示中，使用了“人类感知时间”。
- **核心差异**：这是一种更符合日常交流习惯的“模糊表达”，而非严格的数学换算。

总结：时间的人性化展示，本质上是一个“产品设计”问题。

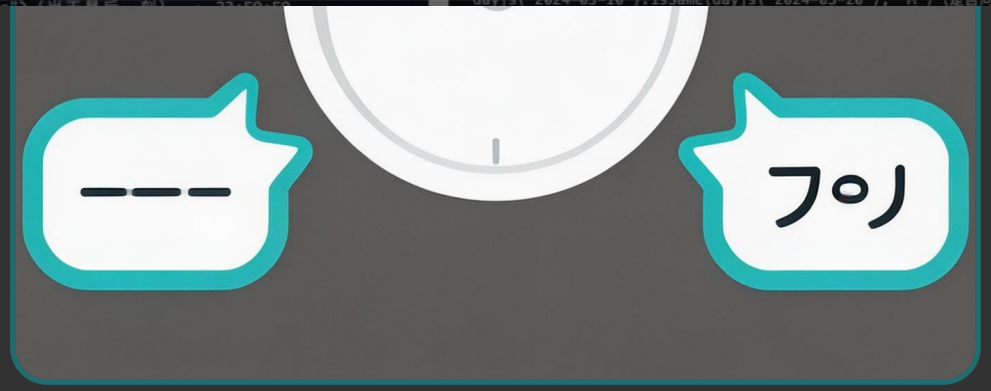
Q3: 为什么是“几秒前”?

```
7. 补充示例（边界与其它 API）
dayjs("2024-03-31").subtract(1, "M") => 2024-02-29
dayjs("2024-02-29").add(1, "y") => 2025-02-28
dayjs("2024-01-15").add(1, "Q") => 2024-04-15
dayjs("2024-01-01").isBefore("2024-06-01") => true
dayjs("2024-06-01").isAfter("2024-01-01") => true
dayjs("2024-03-10").isSame(dayjs("2024-03-20"), "M")（是否同月） => true
dayjs("2024-06-15 12:00:00").startOf("d").format("HH:mm:ss")（当天 0 点） => 00:00:00
dayjs("2024-06-15 12:00:00").endOf("d").format("HH:mm:ss")（当天最后一刻） => 23:59:59
dayjs(Date.now() - 30000).fromNow()（约 30 秒前） => 30 秒前
dayjs(Date.now() - 55000).fromNow()（约 55 秒前，relativeTime 阈值内多显示为 1 分钟） => 1 分钟前
dayjs("2024-01-01").unix()（秒级 Unix 时间戳） => 1704067200

演示结束（可与 dayjs 官方 API 对照着讲设计取舍）
```

```
7. 补充示例（边界与其它 API）
dayjs("2024-03-31").subtract(1, "M") => 2024-02-29
dayjs("2024-02-29").add(1, "y") => 2025-02-28
dayjs("2024-01-15").add(1, "Q") => 2024-04-15
dayjs("2024-01-01").isBefore("2024-06-01") => true
dayjs("2024-06-01").isAfter("2024-01-01") => true
dayjs("2024-03-10").isSame(dayjs("2024-03-20"), "M")（是否同月） => true
dayjs("2024-06-15 12:00:00").startOf("d").format("HH:mm:ss")（当天 0 点） => 00:00:00
dayjs("2024-06-15 12:00:00").endOf("d").format("HH:mm:ss")（当天最后一刻） => 23:59:59
dayjs(Date.now() - 30000).fromNow()（约 30 秒前） => 几秒前
dayjs(Date.now() - 55000).fromNow()（约 55 秒前，relativeTime 阈值内多显示为 1 分钟） => 1 分钟前
dayjs("2024-01-01").unix()（秒级 Unix 时间戳） => 1704038400
```

```
// 英文 locale 配置 ls: 'a few seconds ago'
```



国际化核心洞察

国际化不仅是翻译，更是对不同文化背景下表达习惯的深度适配。

06

总结

SUMMARY

总结与回顾

核心本质

- Parse (解析)：字符串转时间
- Manipulate (操作)：加减/比较
- Format (格式化)：时间转字符串

复杂挑战

- 时区 (Time Zone)：全球时差换算
- 夏令时 (DST)：季节性时间调整
- 国际化 (I18n)：多语言/历法支持

技术参考

- Date
- Day.js：更现代更优雅的设计
- ISO 8601 时间标准

Q&A

THANK YOU

FOR WATCHING

豆包AI生成